

CernVM-FS Filebundles for low-latency software distribution in interactive use cases

Valentin Volk^{1,*}, Georgios Christodoulis¹, and Jakob Blomer¹

¹CERN, 1211 Geneva 23, Switzerland

Abstract. CernVM-FS is a read-only, on-demand file system used to distribute scientific software to the Worldwide LHC Computing Grid and many communities beyond high-energy physics. Its lazy, per-file fetching is highly efficient for batch computing, but it adds noticeable latency to interactive sessions: starting an application that opens hundreds of small files on a cold cache serializes one network round trip after another. We present *filebundles*, a mechanism that exploits the predictable file-access pattern at application startup. A small bundle specification, published next to a trigger file, declares the set of files needed immediately after the trigger is opened. When a client opens the trigger, CernVM-FS prefetches the listed dependencies concurrently using a pool of worker threads, replacing a long serial chain of requests with a short parallel prefetch phase. On a ROOT startup benchmark, filebundles match local-disk performance and keep startup time nearly flat as network latency grows, where the on-demand baseline degrades linearly. The feature is available in CernVM-FS 2.14.

1 Introduction

CernVM-FS [1] is a global, read-only file system for the distribution of scientific software. Rather than downloading and unpacking a full software stack in advance, clients access it directly from a globally available namespace under `/cvmfs`. Files are fetched only when they are actually opened; downloading, decompression and caching all happen transparently. For the user this resembles a streaming service for scientific software: usage can start immediately, and only the parts that are needed are ever transferred.

CernVM-FS is a core component of the Worldwide LHC Computing Grid, but it is also widely used beyond high-energy physics, for example by Euclid, LIGO, LSST, SKA and EESSI. The infrastructure spans on the order of fifteen Stratum 1 mirror servers across Europe, the Americas and Asia. Through `/cvmfs`, clients can access on the order of 3.3×10^{10} files and roughly 2 PB of data, with proven scaling to much larger installations, and more than 8000 container images are served, with the number of repositories continuing to grow.

This lazy, content-addressed model is extremely effective for distributed batch computing, where the benefits of a shared software distribution system dominate any per-file overhead. In interactive sessions, however, the same on-demand behaviour can make application startup noticeably slower than from local disk. This paper presents *filebundles*, a mechanism that reduces that startup latency by prefetching the files an application needs immediately after launch.

*e-mail: valentin.volk@cern.ch

2 Motivation

At the CernVM Users Workshop in 2022, Chris Burr (LHCb) highlighted a practical limitation: in interactive sessions, CernVM-FS can be roughly three to five times slower than local disk, even with a hot cache. For distributed batch computing this is not a real concern, but in interactive work startup latency matters: if opening an analysis environment takes too long, the user experience suffers.

The effect is easy to reproduce on a controlled testbed. Installing ROOT from a release tarball on local disk, importing ROOT in Python takes about 2.47 s. Using the same software stack from `/cvmfs` with a cold cache, the time increases to about 4.69 s. In this setup the network path is very short, so the slowdown is not primarily a bandwidth problem. It confirms that a significant part of the delay comes from the way CernVM-FS serves many files on demand.

2.1 Where does the time go?

When an application such as ROOT starts, it opens many files one after another. On a cold cache, for each file CernVM-FS handles a FUSE upcall, issues an HTTP request, waits for the round trip, downloads the object, decompresses it and returns the data. This per-file overhead is paid again and again, and because the requests are dependent they are effectively serialized. The total amount of work is unremarkable, but the serialized per-file latency adds up. In batch computing this trade-off is acceptable; in interactive use the serialization is precisely what hurts.

It is useful to recall the CernVM-FS cache hierarchy (Fig. 1) in which any optimization has to operate. At the top is the Stratum 0, the writable origin managed by the repository maintainer (“librarian”). Below it are the globally distributed Stratum 1 read-only mirrors. Sites typically add a Squid forward proxy to absorb repeated requests from many clients. On each worker node or workstation CernVM-FS keeps a local disk cache, and above that the Linux kernel maintains the page cache. Filebundles operate within this existing hierarchy. They do not change the data model; they only change *when* and *how* the first wave of file requests is issued.

3 Filebundles

The central idea of filebundles is to exploit information we already have about startup behaviour. For applications like ROOT or Python, the set of files needed immediately after opening a given *trigger* file is often fairly predictable. Instead of discovering those dependencies one file at a time, we describe them in advance.

A bundle is declared by a specification file published next to its trigger file in the same directory, named `.cvmfsbundle-<filename>`, where `<filename>` is the trigger. The specification is a small, versioned JSON document holding minimal metadata and a flat list of dependency paths, expressed as absolute paths from the repository root. A representative ROOT bundle, keyed on the Python package `__init__.py`, contains roughly 250 paths:

```
##CVMFS_BUNDLE version=1 encoding=UTF-8
{
  "name": "CVMFS_BUNDLE",
  "version": "1.0.0",
  "encoding": "UTF-8",
  "dependencies": [
    "/root/lib/libCore.so",
    "/root/lib/libRIO.so",
```

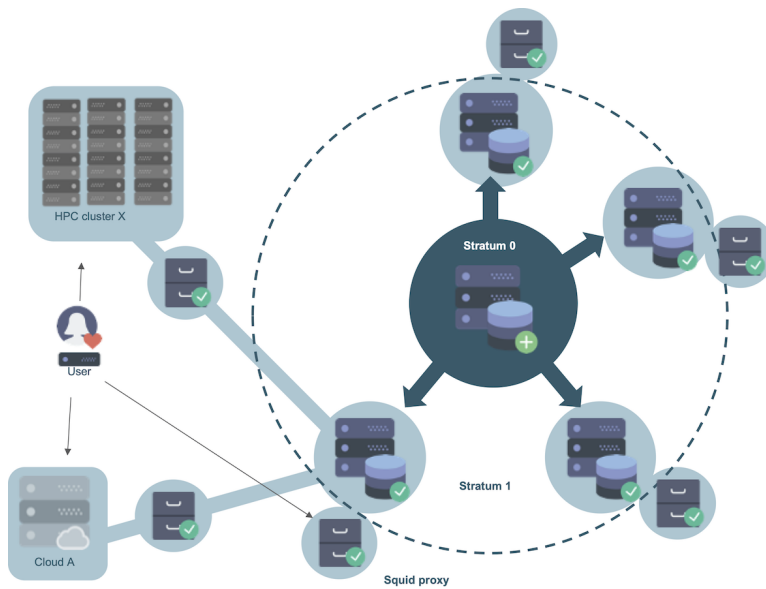


Figure 1. The CernVM-FS cache hierarchy: a writable Stratum 0 origin, globally distributed Stratum 1 mirrors, site-level Squid proxies, and per-client disk and kernel page caches. Filebundles change only the timing of the first wave of requests, not the data model.

```

    "/root/lib/libcpcpp.so",
    "/root/lib/ROOT/_application.py",
    ... (~250 paths total)
]
}

```

When a client opens the trigger file, CernVM-FS detects the matching `.cvmfsbundle` specification and begins prefetching the listed dependencies concurrently, using a pool of worker threads, while the application continues to run. In effect, a long serial chain of dependent requests is replaced by a much shorter parallel prefetch phase that warms the local cache ahead of the application's subsequent `open()` calls.

The feature is disabled by default and is enabled per mount through the client configuration option `CVMFS_PREFETCH_FILEBUNDLES=yes`. The decision whether bundle prefetching is active is taken once at mount time, so clients that leave the option disabled incur no additional overhead. The size of the prefetch thread pool is controlled through `CVMFS_BUNDLE_POOL_SIZE`.

3.1 Publishing a filebundle

From the librarian's point of view the workflow is straightforward. First, choose a suitable trigger file, for example a Python `__init__.py` or a launcher binary such as `root.exe`. Second, determine which files are typically read after that trigger; this can be done by hand or with tracing tools such as `strace` or the built-in `CVMFS_TRACEFILE` mechanism. Third, publish the resulting `.cvmfsbundle-<filename>` specification next to the trigger file, just like any other file in the repository. Removing the specification (and republishing) removes the bundle, after which the trigger behaves like an ordinary file.

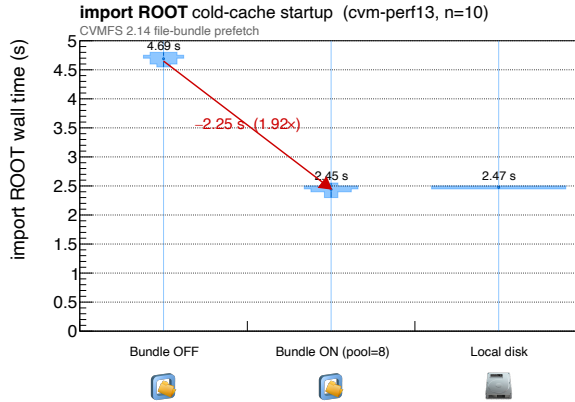


Figure 2. Distribution of ROOT startup times. With filebundles enabled (CVMFS_PREFETCH_FILEBUNDLES=ON), CernVM-FS startup matches local-disk performance, closing the gap to the on-demand cold-cache baseline.

4 Results

Unless noted otherwise, the benchmarks below use ROOT 6.36.12 deployed in a test CernVM-FS repository served from localhost, on an AMD EPYC 7302 system (2×16 cores, 64 threads at ~3.0 GHz, 251 GB RAM) running AlmaLinux 10.1 with kernel 6.12. Serving from localhost isolates the per-file software overhead from external network effects.

4.1 Startup performance

Figure 2 compares the distribution of ROOT startup times for local disk, CernVM-FS on a cold cache without filebundles, and CernVM-FS with filebundles enabled. The key result is that filebundles match local-disk startup performance. In some configurations they are even marginally faster, because the concurrent prefetch overlaps download and decompression work more effectively than the purely on-demand path.

4.2 Prefetch thread pool

Prefetching is implemented with a pool of worker threads whose size is set by CVMFS_BUNDLE_POOL_SIZE. As shown in Fig. 3, startup performance improves as workers are added but saturates around eight threads, which is also the default. Most of the benefit is therefore obtained with a moderate amount of concurrency and without aggressive tuning.

4.3 Network latency

The benefit is most pronounced under network latency. We artificially injected loopback round-trip latency using `tc qdisc netem`. Without filebundles, startup time grows roughly linearly with the round-trip time, because the application must wait for many small dependent fetches in sequence. With filebundles enabled, startup time stays nearly flat over a substantial range (Fig. 4), because the requests are prefetched concurrently: in this idealized case the round-trip penalty is paid only for a small number of batches rather than once per file.

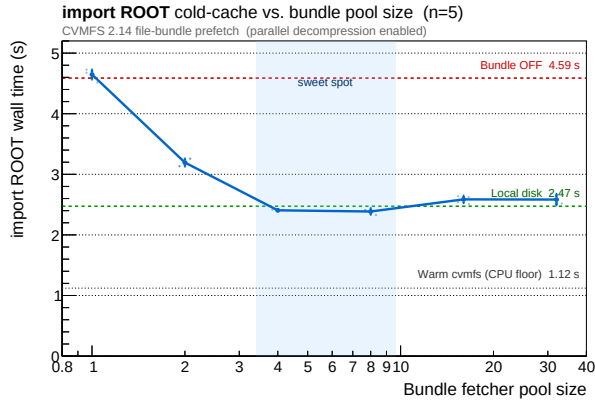


Figure 3. ROOT startup time as a function of the prefetch thread-pool size (CVMFS_BUNDLE_POOL_SIZE). Performance saturates around eight worker threads, the default.

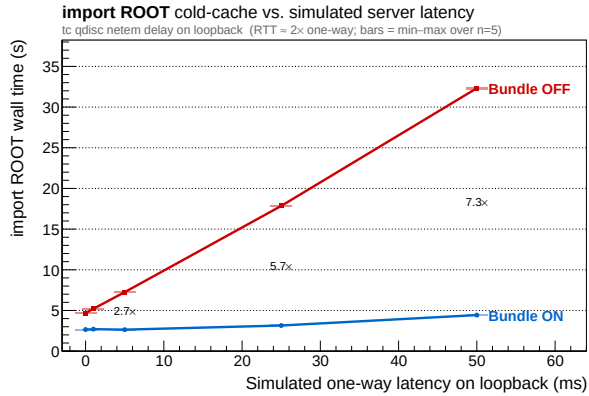


Figure 4. ROOT startup time as a function of injected round-trip latency. The on-demand baseline degrades roughly linearly, while filebundles keep startup nearly flat by overlapping requests.

4.4 Caveats

Filebundles do not reduce the total amount of data transferred; they compress the startup traffic into a short, intense burst. In the benchmark, requests peak at about 1480 per second with filebundles, versus about 680 per second without them (Fig. 5). The feature should therefore be used selectively, in particular on interactive systems such as `lxplus`, where startup latency matters most. Over-prefetching should be avoided, and any rollout should be coordinated with proxy and site administrators so that the request bursts do not overload shared caches.

5 Outlook

Several directions for future work remain. FUSE passthrough, available in CernVM-FS 2.14 on newer kernels, allows direct kernel I/O on cached objects after `open()` and should reduce

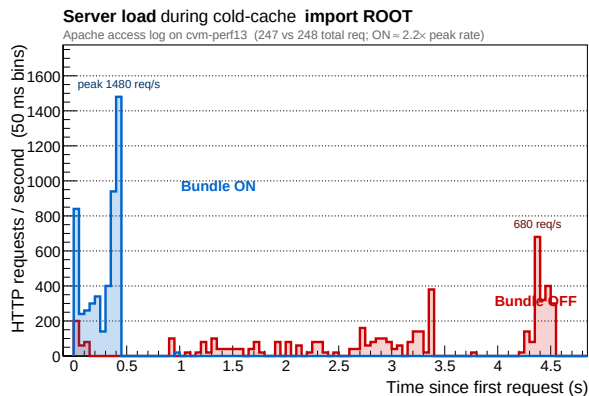


Figure 5. Request rate during startup. Filebundles concentrate the same total traffic into a shorter, more intense burst (peak ~ 1480 req/s versus ~ 680 req/s without bundles).

overhead further; proper benchmarks combining it with filebundles are still needed. For CernVM-FS 2.15 we plan better tooling for filebundle creation, with more automation, more robust updates, and support for specifying file chunks in bundle specifications. Finally, we are considering packfiles that aggregate many small objects in backend storage to reduce request overhead at the source; the main challenge there is backward compatibility.

6 Conclusions

CernVM-FS filebundles are an effective way to accelerate application startup when many files are needed immediately. They require some effort from the librarian – tracing startup behaviour and publishing a `.cvmfsbundle` specification next to a trigger file – but in return they can dramatically reduce cold-start latency, match local-disk startup performance, and remain robust as network latency grows where the on-demand baseline degrades linearly. The feature is available in CernVM-FS 2.14 and is enabled per mount with `CVMFS_PREFETCH_FILEBUNDLES`. Used selectively on interactive systems, it closes much of the gap between a globally distributed, on-demand file system and local disk.

Acknowledgements. We thank Chris Burr (LHCb) for the input that motivated this work, and the CernVM-FS team at CERN.

Disclosure: AI-based tools were used to assist with drafting and formatting this manuscript; all content was reviewed and verified by the authors.

References

- [1] J. Blomer, P. Buncic, R. Meusel, G. Ganis, I. Sfiligoi, D. Thain, The need for a versioned data analysis software environment, in *Proc. of the 9th IEEE International Conference on e-Science* (2013). <https://doi.org/10.1109/eScience.2013.44>
- [2] J. Blomer, C. Aguado-Sánchez, P. Buncic, A. Harutyunyan, Distributing LHC application software and conditions databases using the CernVM file system, *J. Phys.: Conf. Ser.* **331**, 042003 (2011). <https://doi.org/10.1088/1742-6596/331/4/042003>
- [3] CernVM-FS documentation. <https://cvmfs.readthedocs.io/>